Surrogates 7020

Chapter 10: Heteroskedasticity

Dr. Alex Bledar Konomi

Department of Mathematical Sciences
University of Cincinnati

Stochastic Simulations (Computer Models)

- Historically, design and analysis of computer experiments focused on deterministic solvers from the physical sciences via Gaussian process (GP) interpolation (Sacks et al. 1989).
- But nowadays computer modeling is common in the social (Cioffi–Revilla 2014, chap. 8), management (Law 2015) and biological (L. Johnson 2008) sciences, where stochastic simulations abound.
- Queueing systems and agent-based models replace finite elements and simple Monte Carlo (MC) with geometric convergence (Picheny and Ginsbourger 2013).
- Data in geostatistics (Banerjee, Carlin, and Gelfand 2004) and machine learning [ML; Rasmussen and Williams (2006)] aren't only noisy but frequently involve signal-to-noise ratios that are low or changing over the input space.

Stochastic Simulations (Computer Models)

- Noisier simulations/observations demand bigger experiments/studies to isolate signal from noise.
- Demands more sophisticated GP models – not just adding nuggets to smooth over noise, but variance processes to track changes in noise throughout the input space in the face of that heteroskedasticity.
- Partitioning is one option (9.2.2), but leaves something to be desired when underlying processes are smooth and signal-to-noise ratios are low (?).
- A theme in this chapter is that replication, a tried and true design strategy for separating signal from noise, can play a crucial role in computationally efficient heteroskedastic modeling, especially with GPs.

Stochastic Simulations (Computer Models)

- In operations research, stochastic kriging [SK; Ankenman, Nelson, and Staum (2010)] offers approximate methods that exploit large degrees of replication.
- Its independent method of moments-based inferential framework can yield an efficient heteroskedastic modeling capability.
- However the setup exploited by SK is highly specialized; moment-based estimation can strain coherency in a likelihood dominated surrogate landscape.

Stochastic Simulations (Computer Models)

- Here focus is on a heteroskedastic GP technology that offers a modern blend of stochastic kriging and ideas from Machine Learning.
- Besides leveraging replication in design, the idea is: what's good for the mean is good for the variance.
- If it's sensible to model the latent (mean) field with GPs (5.3.2), why not extend that to variance too?
- Unfortunately, latent variance fields aren't as easy to integrate out. (Variances must also be positive, whereas means are less constrained.)
- Numerical schemes are required for variance latent learning, and there are myriad strategies – hence the plethora of ML citations above.

Replication and stochastic kriging

- Replication can be a powerful device for separating signal from noise, offering a pure look at noise not obfuscated by signal.
- Continuing with N-notation from Chapter 9, consider training data pairs $(x_1, y_1), \ldots, (x_N, y_N)$. These make up the "full-N: dataset (X_N, Y_N).
- Now suppose that the number n of unique xi-values in X_N is far smaller than N.
- Let $\bar{Y}_n = (\bar{y}_1, \ldots, \bar{y}_n)^T$ collect averages of a_i replicates at unique locations x_i, and similarly let $\hat{\sigma}_i^2$ collect normalized residual sums of squares.
- Unfortunately, $\bar{Y}_n$ and $\hat{\sigma}_{1:n}^2$ don't comprise of a set of sufficient statistics for GP prediction under the full-N training data.
- But these are nearly sufficient: only one quantity is missing and will be provided momentarily in 10.1.1.

Stochastic kriging mean and variance

Nevertheless "unique-n" predictive equations based on $(\bar{X}_n, \bar{Y}_n)$ are a best linear unbiased predictor (BLUP):
$\mu_n^{SK}(x) = \nu k_n^T(x)(\nu C_n + S_n)^{-1}\bar{Y}_n$
$\sigma_n^{SK}(x)^2 = \nu K_\theta(x,x) - \nu^2 k_n^T(x)(\nu C_n + S_n)^{-1}k_n(x)$,
where $k_n(x) = (K_\theta(x,\bar{x}_1),\ldots,K_\theta(x,\bar{x}_n))^T$ as in Chapter 5,

$$S_n = [\hat{\sigma}^2_{1:n}]A_n^{-1} = Diag(\hat{\sigma}^2_1/a_1,\ldots,\hat{\sigma}^2_n/a_n),$$

$C_n = \{K_\theta(\bar{x}_i,\bar{x}_j)\}_{1\le i,j\le n}$, and $a_i >> 1$.
This is the basis of stochastic kriging [SK; Ankenman, Nelson, and Staum (2010)], implemented as an option in DiceKriging (Roustant, Ginsbourger, and Deville 2018) and *mlegp* (Dancik 2018) packages on CRAN. **for one observation** $\nu = \tau^2$

Stochastic kriging mean and variance

- An independent moments-based estimate of variance is used in lieu of the more traditional, likelihood-based (hyperparametric) alternative.
- This could be advantageous if variance is changing in the input space, as discussed further in 10.2.
- Computational expedience is readily apparent even in the usual homoskedastic setting:

$$S_n = Diag(\frac{1}{n}\sum_{i=1}^{n}\sigma_i^2).$$

Only $O(n^3)$ matrix decompositions are required, which could represent a huge savings compared with $O(N^3)$ if the degree of replication is high.

Stochastic kriging mean and variance

Independent calculations have their drawbacks.

- Thinking heteroskedastically, this setup lacks a mechanism for specifying a belief that variance evolves smoothly over the input space.
- Not neccessary good for predictions in unobserved x.
- Ankenman, Nelson, and Staum (2010) suggest smoothing σ_i^2-values with a second GP, but that two-stage approach will feel ad hoc.
- and numbers of replicates ai must be relatively large in order for the σ_i^2-values to be reliable. $(a_i > 10)$.
- In what follow we will give a more unified form and also less ad-hoc.

Woodbury trick

Let D and B be invertible matrices of size $N \times N$ and $n \times n$, respectively, and let U and V^T be matrices of size $N \times n$. The Woodbury identities are:

$$(D + UBV)^{-1} = D^{-1} - D^{-1}U(B^{-1} + VD^{-1}U)^{-1}VD^{-1} \quad (1)$$

$$|D + UBV| = |B^{-1} + VD^{-1}U| \times |B| \times |D| \quad (2)$$

To build $K_N = UC_nV + D$ for GPs under replication, take $U = V^T = Diag(1_{a_1}, \ldots, 1_{a_n})$, where 1_k is a k-vector of ones, $D = gI_N$ with nugget g, and $B = C_n$. Later, we'll take $D = \Lambda_n$ where Λ_n is a diagonal matrix of latent variances.

EI search function

```
n <- 50
Xn <- matrix(runif(2*n), ncol=2)
Xn <- Xn[order(Xn[,1]),]
ai <- c(8, 27, 38, 45)
mult <- rep(1, n)
mult[ai] <- c(25, 30, 9, 40)
XN <- Xn[rep(1:n, times=mult),]
N <- sum(mult)
##
library(hetGP)
KN <- cov_gen(XN, theta=0.2)
Kn <- cov_gen(Xn, theta=0.2)
##
U <- c(1, rep(0, n - 1))
for(i in 2:n){
 tmp <- rep(0, n)
 tmp[i] <- 1
 U <- rbind(U, matrix(rep(tmp, mult[i]), nrow=mult[i], byrow=TRUE))
}
U <- U[,n:1]  ## for plotting in a moment
```

Plot Covariances and U

```
cols <- heat.colors(128)
layout(matrix(c(1, 2, 3), 1, 3, byrow=TRUE), widths=c(2, 1, 2))
image(Kn, x=1:n, y=1:n, main="uniq-n: Kn", xlab="1:n", ylab="1:n", col=cols)
image(t(U), x=1:n, y=1:N, asp=1, main="U", xlab="1:n", ylab="1:N", col=cols)
image(KN, x=1:N, y=1:N, main="full-N: KN", xlab="1:N", ylab="1:N", col=cols)
```

Postrior Mean and Variance

$$\nu k_N^T(x)(\nu C_N + S_N)^{-1} Y_N = k_n^T(x)(C_n + \Lambda_n A_n^{-1})^{-1} \bar{Y}_n$$

$$vk - \nu^2 k_N^T(x)(\nu C_N + S_N)^{-1} k_N(x) = vk - \nu k_n^T(x)(C_n + \Lambda_n A_n^{-1})^{-1} k_n,$$

where $vk = \nu K_\theta(x, x)$ is used as a shorthand to save horizontal space.

Postrior Mean and Variance

- In words, typical full-N predictive quantities may be calculated identically through unique-n counterparts, potentially yielding dramatic savings in computational time and space.
- The unique-n predictor, implicitly defining $mu_n(x)$ and $\sigma_n^2(x)$ on the right-hand side(s) and updating SK (10.1), is unbiased and minimizes mean-squared prediction error (MSPE) by virtue of the fact that those properties hold for the full-N predictor on the left-hand side(s).
- No asymptotic or frequentist arguments are required. Crucially, no minimum data or numbers of replicates (e.g., $a_i \geq 10$ for SK) are required to push asymptotic arguments through, although replication can still be helpful from a statistical efficiency perspective.

Likelihood

The same trick can be played with the concentrated log likelihood. Recall that

$$K_n = (C_n + \Lambda_n A_n^{-1})^{-1},$$

where for now $\Lambda_n = gI_n$ encoding homoskedasticity. Later I shall generalize$\Lambda_n = Diag(\lambda_1, \ldots, \lambda_n)$ for the heteroskedastic setting.

$$l = c - N^2 \log \hat{\nu}_N - \frac{1}{2} \sum_{i=1}^{n} [(a_i - 1) \log \lambda_i + \log a_i] - \frac{1}{2} \log |K_n|. \quad (3)$$

(where $\hat{\nu}_N = \frac{1}{N}(Y_N^T \Lambda_N^{-1} Y_N - \bar{Y}_n^T A_n \Lambda_n^{-1} \bar{Y}_n + \bar{Y}_n^T K_n^{-1} \bar{Y}_n)$. Notice additional terms in $\hat{\nu}_N$ compared with $\hat{\nu}_n = \frac{1}{n} \bar{Y}_n^T K_n^{-1} \bar{Y}_n$. Since Λ_N is diagonal, evaluation of l requires just $O(n^3)$ operations beyond the $O(N)$ required for $\bar{Y}_n$.

Likelihood

The trick is to use the Woodbury formula above to decompose:

$$Y_N^T(C_N+\Lambda_N)Y_N = Y^T\Lambda_N^{-1}Y - \bar{Y}^T A_n \Lambda_n^{-1}\bar{Y} + \bar{Y}^T(C_n+\Lambda_n A_n^{-1})^{-1}\bar{Y}$$

$$\log|C_N+\Lambda_N| = \log|C_n+\Lambda_n A_n^{-1}| + \sum_{i=1}^{n}[(a_i-1)\log\lambda_i + \log a_i]$$

Where $\Lambda_N = \frac{1}{\nu}S_N$ To maximize the likelihood, take derivatives in respect to the hyper-paramters.

Computational Advantage

```
library(lhs)
Xbar <- randomLHS(100, 2)
Xbar[,1] <- (Xbar[,1] - 0.5)*6 + 1
Xbar[,2] <- (Xbar[,2] - 0.5)*6 + 1
ytrue <- Xbar[,1]*exp(-Xbar[,1]^2 - Xbar[,2]^2)
a <- sample(1:50, 100, replace=TRUE)
N <- sum(a)
X <- matrix(NA, ncol=2, nrow=N)
y <- rep(NA, N)
nf <- 0
for(i in 1:100) {
 X[(nf+1):(nf+a[i]),] <- matrix(rep(Xbar[i,], a[i]), ncol=2, byrow=TRUE)
 y[(nf+1):(nf+a[i])] <- ytrue[i] + rnorm(a[i], sd=0.01)
 nf <- nf + a[i]
}
############################
Lwr <- rep(sqrt(.Machine$double.eps), 2)
Upr <- rep(10, 2)
fN <- mleHomGP(list(X0=X, Z0=y, mult=rep(1, N)), y, Lwr, Upr)
un <- mleHomGP(X, y, Lwr, Upr)
c(fN=fN$time, un=un$time)
rbind(fN=fN$theta, un=un$theta)
```

Hetroscedasticity

- Heteroskedastic GP modeling targets learning the diagonal matrix Λ_N, or its unique-n counterpart Λ_n; see Eq. (10.4). Allowing λ_i-values to exhibit heterogeneity, i.e., not all $\lambda_i = g/\nu$ as in the homoskedastic case, is easier said than done.
- Care is required when performing inference for such a high-dimensional parameter. SK (10.1) suggests taking $\Lambda_n A^{-1} = S_n = Diag(\sigma_1^2, \ldots, \sigma_n^2)$, but that requires large numbers of replicates a_i and is anyways useless out of sample. By fitting each σ_i^2 separately there's no pooling effect for interpolation/smoothing.
- Ankenman, Nelson, and Staum (2010) suggest the quick fix of fitting a second GP to the variance observations with "data":

$$(\bar{x}_1, \hat{\sigma}_1^2), \ldots, (\bar{x}_n, \hat{\sigma}_n^2),$$

to obtain a smoothed variance for use out of sample.

Hetroscedasticity

- A more satisfying approach that's similar in spirit, coming from ML (Goldberg, Williams, and Bishop 1998) and actually predating SK by more than a decade, applies regularization that encourages a smooth evolution of variance.
- Goldberg et al. introduce latent (log) variance variables under a GP prior and develop an MCMC scheme performing joint inference for all unknowns, including hyperparameters for both mean and noise GPs.
- The overall method, which is effectively on the order of $O(TN^4)$ for T MCMC samples, is totally impractical but works well on small problems.
- Several authors have economized on aspects of this framework (Kersting et al. 2007; Lazaro–Gredilla and Titsias 2011) with approximations and simplifications of various sorts, (but none of these have resulted in public R software).

Latent Variable Approach

- The key ingredient in these works, of latent variance quantities smoothed by a GP, has merits and can be effective when handled gingerly. Binois, Gramacy, and Ludkovski (2018) introduced the methodology described below.
- Let $\delta_1, \delta_2, \ldots, \delta_n$ denote latent variance variables (or equivalently latent nuggets), each corresponding to one of the n unique design sites $\bar{x}_i$ under study.
- It's important to introduce latents only for the unique-n locations. A similar approach in the full-N setting, i.e., without exploiting Woodbury identities, is fraught with identifiability and numerical stability challenges.

Latent Variable Formulation

Store these latents diagonally in a matrix Δ_n and place them under a GP prior:

$$\Delta_n \sim N_n(0, \nu_{(\delta)}(C_{(\delta)} + g_{(\delta)}A_n^{-1})). \tag{4}$$

- Inverse exponentiated squared Euclidean distance-based correlations in $n \times n$ matrix $C_(\delta)$ are hyperparameterized by novel lengthscales $\theta_{(\delta)}$ and nugget $g_{(\delta)}$.
- Smoothed λ_i-values can be calculated by plugging $Delta_n$ into GP mean predictive equations (**Posterior Mean of** δ).
- A positive nonzero mean $\mu_{(\delta)}$ may be preferable for variances. The hetGP package automates the GLM mean.

Latent Variable Formulation

Variances must be positive, and the equations above give nonzero probability to negative δ_i and λ_i-values.

- One solution is to threshold latent variances at zero.
- Another is to model $\log \Delta_n$ in this way instead. The latter option, guaranteeing positive variance after exponentiating, is the default in hetGP but both variations are implemented.
- Differences in empirical performance and mathematical development are slight.
- Modeling $\log \Delta_n$ makes derivative expressions a little more complicated after applying the chain rule, so our exposition here shall continue with the simpler un-logged variation. For more see exercise in 10.4.

Joint Likelihood

Rather than cumbersome MCMC, Binois et al. describe how to stay within a (Woodbury) MLE framework, by defining a joint log likelihood over both mean and variance GPs:

$$l = c - N^2 \log \hat{\nu}_N - \frac{1}{2} \sum_{i=1}^{n} [(a_i - 1) log \lambda_i + \log a_i] - \frac{1}{2} \log |K_n| \\ - n^2 \log \hat{\nu}_\delta - \frac{1}{2} \log |K_{\delta_n}| \tag{5}$$